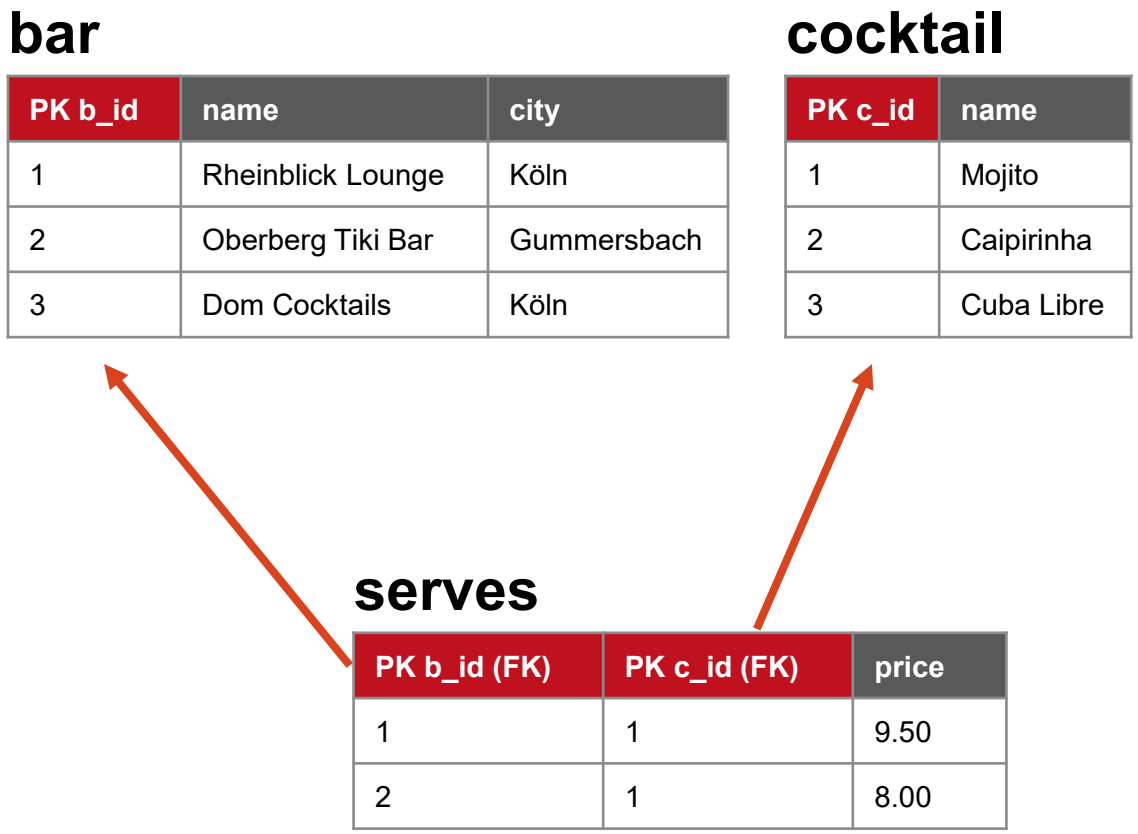
# Standard SQL Commands

How to create, fill, query and change tables, using the bar & cocktail example



# Four Families of SQL Commands

## DDL

Define structure

| command |
| --- |
| `CREATE` |
| `ALTER` |
| `DROP` |

## DML

Change data

| command |
| --- |
| `INSERT` |
| `UPDATE` |
| `DELETE` |

## DQL

Query data

| command |
| --- |
| `SELECT` |
| `+ WHERE, JOIN,` |
| `GROUP BY, ORDER BY` |

## DCL / TCL

Rights & transactions

| command |
| --- |
| `GRANT, REVOKE` |
| `COMMIT` |
| `ROLLBACK` |

**DDL:** Data Definition Language builds and changes tables themselves.
**DML / DQL:** Data Manipulation and Query Language work on the rows inside them.
**This deck:** the red families, step by step, on the bar / cocktail / serves tables.

# CREATE TABLE Builds the Structure

```
CREATE TABLE bar (
  b_id  INT PRIMARY KEY,
  name  VARCHAR(50) NOT NULL,
  city  VARCHAR(50)
);

CREATE TABLE serves (
  b_id  INT REFERENCES bar(b_id),
  c_id  INT REFERENCES cocktail(c_id),
  price DECIMAL(5,2),
  PRIMARY KEY (b_id, c_id)
);
```

**bar**

| PK b_id | name | city |
|---|---|---|
| | | |

**serves**

| PK b_id (FK) | PK c_id (FK) | price |
|---|---|---|
| | | |

**Columns:** name + data type (INT, VARCHAR, DECIMAL …)
**Constraints:** PRIMARY KEY, NOT NULL, REFERENCES (= foreign key)
**Result:** an empty table, ready for rows

# INSERT INTO Adds Rows

```
INSERT INTO bar (b_id, name, city)
VALUES (1, 'Rheinblick Lounge', 'Köln'),
       (2, 'Oberberg Tiki Bar', 'Gummersbach'),
       (3, 'Dom Cocktails', 'Köln');

INSERT INTO serves VALUES (1, 1, 9.50);
```



**bar**

| PK b_id | name | city |
|---|---|---|
| 1 | Rheinblick Lounge | Köln |
| 2 | Oberberg Tiki Bar | Gummersbach |
| 3 | Dom Cocktails | Köln |

**Order matters:** values must match the column list in number, order and type.
**Keys are checked:** a second row with b_id 1 is rejected (primary key must be unique).
**Foreign keys too:** INSERT INTO serves VALUES (4, 1, 7.00) fails, because bar 4 doesn't exist.

# SELECT … WHERE Reads and Filters

```
SELECT name, city
FROM   bar
WHERE  city = 'Köln';
```

**SELECT** chooses the columns (* = all)
**FROM** names the table
**WHERE** keeps only rows where the condition is true
**Operators:** = <> < > BETWEEN LIKE IN IS NULL, combined with AND / OR

## bar

| PK b_id | name | city |
|---|---|---|
| **1** | **Rheinblick Lounge** | **Köln** |
| 2 | Oberberg Tiki Bar | Gummersbach |
| **3** | **Dom Cocktails** | **Köln** |



## result

| name | city |
|---|---|
| Rheinblick Lounge | Köln |
| Dom Cocktails | Köln |

# GROUP BY Summarises Rows

```
SELECT   b_id,
         COUNT(*)   AS cocktails,
         AVG(price) AS avg_price
FROM     serves
GROUP BY b_id
HAVING   COUNT(*) >= 2
ORDER BY avg_price DESC;
```

**serves**

| PK b_id (FK) | PK c_id (FK) | price |
|---|---|---|
| 1 | 1 | 9.50 |
| 1 | 2 | 9.00 |
| 2 | 1 | 8.00 |
| 3 | 2 | 10.00 |
| 3 | 3 | 9.00 |

**result**

| b_id | cocktails | avg_price |
|---|---|---|
| 3 | 2 | 9.50 |
| 1 | 2 | 9.25 |

**Aggregates:** COUNT, SUM, AVG, MIN, MAX
**GROUP BY** makes one row per group
**WHERE vs HAVING:** WHERE filters rows before grouping, HAVING filters groups after
**ORDER BY** sorts the result (ASC default, DESC)

Bar 2 has only one cocktail, so HAVING drops it.

# JOIN Follows the Keys

```
SELECT b.name AS bar,
       c.name AS cocktail,
       s.price
FROM   serves s
JOIN   bar b      ON s.b_id = b.b_id
JOIN   cocktail c ON s.c_id = c.c_id;
```

**result**

| bar | cocktail | price |
|---|---|---|
| Rheinblick Lounge | Mojito | 9.50 |
| Rheinblick Lounge | Caipirinha | 9.00 |
| Oberberg Tiki Bar | Mojito | 8.00 |
| Dom Cocktails | Caipirinha | 10.00 |
| Dom Cocktails | Cuba Libre | 9.00 |

**ON** matches each foreign key to the primary key it references
**INNER JOIN** (plain JOIN) keeps only rows with a match on both sides
**LEFT JOIN** keeps every row of the left table, filling gaps with NULL

IDs from serves become readable names from bar and cocktail.

# UPDATE and DELETE Change Rows

## UPDATE

```
UPDATE serves
SET    price = 10.00
WHERE  b_id = 2 AND c_id = 1;
```

| PK b_id (FK) | PK c_id (FK) | price |
|---|---|---|
| **2** | **1** | **8.00 → 10.00** |

## DELETE

```
DELETE FROM serves
WHERE  b_id = 3 AND c_id = 3;
```

| PK b_id (FK) | PK c_id (FK) | price |
|---|---|---|
| ~~3~~ | ~~3~~ | ~~9.00~~ |

**Always use WHERE:** without it, UPDATE changes and DELETE removes every row in the table.
**Foreign keys protect you:** DELETE FROM bar WHERE b_id = 1 fails while serves still points to bar 1 (unless ON DELETE CASCADE is set).
**Tip:** run the same WHERE as a SELECT first to see which rows will be hit.

# ALTER and DROP Change the Structure

```
ALTER TABLE bar ADD COLUMN opened INT;
ALTER TABLE bar DROP COLUMN opened;

DROP TABLE serves;   -- table + data gone
```

## bar after ADD COLUMN

| PK b_id | name | city | opened |
|---|---|---|---|
| 1 | Rheinblick Lounge | Köln | NULL |
| 2 | Oberberg Tiki Bar | Gummersbach | NULL |
| 3 | Dom Cocktails | Köln | NULL |

## Which one removes what?

| command | rows | table |
|---|---|---|
| DELETE FROM t | some / all | kept |
| TRUNCATE TABLE t | all | kept |
| DROP TABLE t | all | removed |

**ALTER TABLE** adds, removes or renames columns and constraints on an existing table.
**DROP TABLE** can't be undone outside a transaction.
Drop serves before bar, because serves references it.

# How the Database Runs a SELECT

**You write it in this order:**

```
SELECT … FROM … JOIN … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT
```

**The database processes it in this order:**




**Why it matters:** an alias defined in SELECT (avg_price) can't be used in WHERE, which runs earlier, but works in ORDER BY, which runs later.